As classes apresentadas neste artigo demonstram como a programação em Java consegue manipular as datas, horas e internacionalizá-las. No Java o tempo é representado em milissegundos, sendo medido a partir da data 01/01/1970. Essa data pode ser recuperada através do comando `System.currentTimeMillis()`.

Abaixo são mostradas as principais classes utilizadas quando se trata de algum dado que envolva datas.

Classe Date

A data representa o tempo, um tempo é composto por ano, mês, dia atual, minuto atual, entre outras propriedades que essa classe possui.

Hoje a maioria dos métodos da classe Date estão classificados como deprecated (depreciados), ou seja, são métodos que não são mais utilizados, por isso essa classe foi substituída pela classe `Calendar`, para haver suporte correto à internacionalização do sistema de datas.

Saiba mais: Classe Calendar

Na **Listagem 1** é instanciada a classe Date e exibida a data atual em detalhes.

```
import java.util.Date;

public class Testa_Date {
  public static void main(String[] args) {
    Date data = new Date();
    System.out.println("Data Agora: "+data);
  }
}
```

Listagem 1. Testando a classe Date

Classe Calendar

Essa classe pode produzir os valores de todos os campos de calendário necessários para implementar a formatação de data e hora, para uma determinada língua e estilo de calendário. Por exemplo, japonês, americano, italiano, brasileiro entre outros.

A classe Calendar é a mais usada quando se trata de datas, mas como é uma classe abstrata, não pode ser instanciada, portanto para obter um calendário é necessário usar o método estático `getInstance()`, apresentado no exemplo da **Listagem 2**.

```
import java.util.Calendar;

public class Data_Calendar{
```

```
  public static void main(String[] args) {
    Calendar c = Calendar.getInstance();
    System.out.println("Data e Hora atual: ”+c.getTime());
  }
}
```

Listagem 2. Recuperação da data com a classe Calendar

Com a classe Calendar também se consegue manipular a data e hora com os métodos que são fornecidos, abaixo seguem os exemplos.

```
import java.util.Calendar;

public class Testa_Metodo_Get_Calendar{

  public static void main(String[] args) {
    Calendar c = Calendar.getInstance();

    System.out.println("Data/Hora atual: "+c.getTime());
    System.out.println("Ano: "+c.get(Calendar.YEAR));
    System.out.println("Mês: "+c.get(Calendar.MONTH));
    System.out.println("Dia do Mês: "+c.get(Calendar.DAY_OF_MONTH));
  }
}
```

Listagem 3. Mostra o dia da semana, mês e ano

```
import java.util.Calendar;

public class Testa_Metodos_Set_Get_Calendar{

  public static void main(String[] args) {
    Calendar c = Calendar.getInstance();
    c.set(Calendar.YEAR, 1995);
    c.set(Calendar.MONTH, Calendar.MARCH);
    c.set(Calendar.DAY_OF_MONTH, 20);

    System.out.println("Data/Hora atual: "+c.getTime());
    System.out.println("Ano: "+c.get(Calendar.YEAR));
    System.out.println("Mês: "+c.get(Calendar.MONTH));
    System.out.println("Dia do Mês: "+c.get(Calendar.DAY_OF_MONTH));
  }

}
```

Listagem 4. Alterando a data/hora com método set

Na **Listagem 5**, mostra-se como recuperar a hora do dia, fazendo uma interação com o usuário informando uma mensagem de boas vindas.

```
import java.util.Calendar;

public class Msg_Boas_Vindas_Calendar{

  public static void main(String[] args) {
    Calendar c1 = Calendar.getInstance();
    int hora = c1.get(Calendar.HOUR_OF_DAY);

if(hora > 6 && hora < 12){
      System.out.println("Bom Dia");
    }else if(hora > 12 && hora < 18){
      System.out.println("Boa Tarde");
    }else{
      System.out.println("Boa Noite");
    }
  }
}
```

Listagem 5. Recuperando a hora do dia

Para mais informações sobre a classe Calendar, acesse o link para as Documentações Oracle.

DateFormat

Essa classe permite converter informações do tipo String para data do tipo Date, permitindo seguir um formato. Consegue-se trabalhar ao contrário, convertendo um dado do tipo Date para uma String. Por ser uma classe abstrata, não é possível instanciá-la, por isso deve ser usado para método estático `getDateInstance()`. Sempre quando declarado é preciso importar o pacote `java.text`.

Abaixo são mostrados alguns exemplos dos métodos de formatação das datas.

```
import java.util.Calendar;
import java.text.DateFormat;
import java.util.Date;

public class Formatando_Datas{

  public static void main(String[] args) {
    Calendar c = Calendar.getInstance();
```

```
        c.set(2013, Calendar.FEBRUARY, 28);
        Date data = c.getTime();
        System.out.println("Data atual sem formatação: "+data);

        //Formata a data
        DateFormat formataData = DateFormat.getDateInstance();
        System.out.println("Data atual com formatação: "+
    formataData.format(data));
```

Listagem 6. Formatando data atual

```
import java.util.Calendar;
import java.text.DateFormat;
import java.util.Date;

public class Formatando_Saida_Datas{

  public static void main(String[] args) {
    Calendar c = Calendar.getInstance();
    Date data = c.getTime();

    DateFormat f = DateFormat.getDateInstance(DateFormat.FULL); //Data
COmpleta
    System.out.println("Data brasileira: "+f.format(data));

f = DateFormat.getDateInstance(DateFormat.LONG);
    System.out.println("Data sem o dia descrito: "+f.format(data));

    f = DateFormat.getDateInstance(DateFormat.MEDIUM);
```

Listagem 7. Formatações de datas

Conversões

Às vezes é preciso transformar um texto em uma data ou vice versa, para isso abaixo é exibida a função `parse` e a classe `SimpleDateFormat`. Geralmente a classe `SimpleDateFormat` é mais usada quando trata-se de formatação de datas, pois já no seu construtor, quando instanciada, permite passar como argumento o formato da data desejada, como apresentada nos exemplos abaixo.

```
import java.util.Calendar;
import java.text.DateFormat;
import java.util.Date;
import java.text.SimpleDateFormat;
import java.text.ParseException;
```

```
public class Conversao_Datas{

  public static void main(String[] args) throws ParseException{
    Calendar c = Calendar.getInstance();
    Date data = c.getTime();
    DateFormat f = DateFormat.getDateInstance();

    Date data2 = f.parse("12/01/1995");
    System.out.println(data2);

    SimpleDateFormat sdf = new SimpleDateFormat("dd/MM/yyyy");
```

Listagem 8. Convertendo Datas com método parse e a classe `SimpleDateFormat`

Para mais detalhes, acesse o link para as Documentações Oracle.

Saiba mais: Boas práticas ao trabalhar com datas

Internacionalização das datas

O Java oferece a classe `Locale`, que permite definir de qual país deseja-se obter o retorno das informações, como data e hora. Nos exemplos acima não foi necessário instanciar essa classe, pois já é detectado automaticamente do computador quais são as configurações regionais. Na **Listagem 9** é mostrado como formatar data e hora de acordo com outros países.

```
import java.util.Calendar;
import java.util.Locale;
import java.text.DateFormat;
import java.text.ParseException;
import java.util.Date;

public class Locale_Datas{

  public static void main(String[] args) throws ParseException {
    Calendar c = Calendar.getInstance();
    Date data = c.getTime();

    Locale brasil = new Locale("pt", "BR"); //Retorna do país e a língua
    Locale eua = Locale.US;
    Locale italia = Locale.ITALIAN;

    DateFormat f2 = DateFormat.getDateInstance(DateFormat.FULL, brasil);
    System.out.println("Data e hora brasileira: "+f2.format(data));
```